

Database-Aware Monolith Decomposition via Multi-Objective Optimization

Anonymous Author(s)

Abstract

The decomposition of monolithic architectures into microservices remains a challenging task, particularly when defining appropriate microservices bounds. While most existing approaches focus on structural dependencies among software components to decompose a monolith, database ownership is often neglected during the process, despite being an important principle of microservice architectures. In this work, monolith decomposition is formulated as a constrained multi-objective optimization problem in which database dependencies are enforced as hard constraints. The proposed approach employs the Non-dominated Sorting Genetic Algorithm III (NSGA-III) to explore trade-offs among structural modularity, inter-service communication, interface complexity, and microservice size balance of the candidate decompositions. In addition, the approach adopts a canonical, partition-based representation for candidate decompositions to avoid redundant encodings of equivalent decompositions. This work builds upon a previously introduced formulation by refining the decomposition representation and providing an empirical evaluation on two monolithic applications commonly used in the literature. The results indicate that the approach is able to identify diverse and feasible decomposition alternatives that respect database isolation constraints and achieve competitive results when compared to non-evolutionary approaches.

CCS Concepts

• **Software and its engineering** → *Search-based software engineering*.

Keywords

monolith decomposition, microservices, evolutionary algorithms, multi-objective optimization, search-based software engineering

ACM Reference Format:

Anonymous Author(s). 2026. Database-Aware Monolith Decomposition via Multi-Objective Optimization. In . ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Monolithic architectures encapsulate all application functionality into a single deployment unit, which simplifies early development but hinders scalability, maintainability, and technological evolution as application grows [10]. Consequently, practitioners migrate

toward microservices architectures, which is composed of a set of finely grained and light-weight components that each define a specific business need and communicate with each other through different communication mechanisms [6]. A key challenge in this migration process is the decomposition of a monolithic application into a set of well-defined microservices, as inadequate decompositions may lead to high inter-service communication, low cohesion, and tightly coupled services.

Monolith decomposition is widely modeled in the literature as a graph partitioning problem with the objective of maximizing cohesion while minimizing coupling among software components. This problem is NP-hard, which limits the applicability of exact solutions in large-scale monolith applications [4].

Among existing decomposition techniques, the majority focus on structural dependencies between software components, such as classes or modules, and their interaction to obtain candidate decompositions. However, microservices architectures define a one-database-per-microservice schema as a design principle [6], which enforces data ownership and service independence. Therefore, omitting database dependencies during decomposition may result in services that share databases and introduce tight coupling at the persistence layer. In this sense, database-aware monolith decomposition remains underexplored [13].

In this work, we build upon a previously introduced formulation of monolith decomposition [12] as a constrained multi-objective optimization problem that incorporates database dependencies as constraints. The problem is interpreted from a bin-packing perspective, where classes are assigned to microservices and database tables impose exclusivity constraints across services.

To explore trade-offs among multiple conflicting quality objectives, we employ the Non-dominated Sorting Genetic Algorithm III (NSGA-III), which supports the identification of alternative solutions under competing criteria. In this setting, we adopt a canonical, partition-based representation in which candidate solutions are defined by class groupings, reducing representational redundancy and aligning with the architectural semantics of the problem.

The proposed approach is evaluated on two monolithic applications and compared against established non-evolutionary baseline decomposition techniques. For each application, a set of architectural metrics is calculated and compared against baseline techniques to evaluate the quality of candidate microservices decompositions. The results indicate that the proposed approach can effectively enforce database isolation while identifying competitive trade-off solutions.

The remainder of this paper is organized as follows. Section 2 reviews related work on monolith decomposition. Section 3 presents the proposed problem formulation, and Section 4 describes the optimization approach. Section 6 reports the experimental results. Finally, Section 7 concludes the study and presents future work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

2 Related Work

Decomposing a monolithic application is the most addressed issue during monolith to microservices migration [13]. For this reason, multiple approaches have been proposed in the literature to tackle it. Most existing decomposition techniques represent the monolithic application as a dependency graph, where nodes model software components such as classes or methods, and edges capture static and/or dynamic dependencies between them. Accordingly, decomposition is most commonly formulated as a clustering or partitioning problem, with the objective of maximizing cohesion while minimizing coupling among the resulting services [1].

A set of approaches is based on clustering and graph-partitioning techniques originally proposed for software modularization. For instance, Bunch [15] applies search-based clustering to derive modular software decompositions based on structural dependencies. However, these approaches were not designed for microservices architectures or monolith decomposition specifically and do not address concerns such as service independence or interaction.

More recent techniques explicitly target monolith decomposition by incorporating additional sources of information. For instance, Mono2Micro [8] and FoSCI [7] use static dependencies with execution traces to derive microservice candidates; CoGCN [3] employs graph neural networks to get microservice boundaries from dependency graphs; MEM [14] integrates both structural and behavioral information to guide decomposition. Although the underlying methodology of these approaches differ, they primarily focus on relationships among software components and their interactions.

Given the NP-hard nature of the monolith decomposition problem, search-based software engineering (SBSE) techniques have also been adopted. Only a limited number of studies formulate monolith decomposition as a multi-objective optimization problem and employ Pareto-based evolutionary algorithms, most notably NSGA-II, to explore trade-offs between structural properties such as cohesion, coupling, service granularity, and communication overhead [7, 16]. Many-objective formulations have also been considered in the context of microservice migration, with one approach based on NSGA-III to generate alternative architecture candidates under multiple competing criteria [2].

However, persistence concerns are not modeled explicitly during the decomposition process. Even approaches that extract database-related artifacts from execution logs or identify data entity classes tend to use this information to support service identification or post-hoc analysis, rather than to restrict the space of admissible decompositions [11]. As a result, database ownership is often assumed to be addressed after microservice decompositions are obtained.

In contrast to existing work, this work considers monolith decomposition as a constrained multi-objective optimization problem in which database ownership is enforced during decomposition. By integrating persistence-related constraints directly into the optimization process, the proposed approach restricts the search space to feasible architectures and supports the exploration of trade-offs among structural and interaction-based objectives under realistic architectural assumptions.

3 Problem Formulation

A monolith application is composed of n classes and a set of T database tables. We define $C = \{c_1, c_2, \dots, c_n\}$ as the set of classes of a monolithic application. Structural dependencies among classes are represented by a directed graph $G = (C, E)$, where an edge $(c_i, c_j) \in E$ denotes a call or dependency from class c_i to class c_j .

Monolith decomposition consists of deriving a set of microservices $M = \{m_1, m_2, \dots, m_k\}$ such that each microservice is a non-empty subset of classes and together form a partition of C :

$$\bigcup_{j=1}^k m_j = C \quad \text{and} \quad m_i \cap m_j = \emptyset \quad \forall i \neq j$$

The number of microservices k is determined by the optimization process. Therefore, each candidate solution corresponds to a partition of the class set.

From a modeling perspective, the decomposition problem can be interpreted as an assignment of classes (items) to containers or microservices (bins). Unlike the classical bin-packing formulation, the number of bins is not fixed and no explicit capacity constraints are imposed. Instead, desirable size distributions and structural properties are encouraged through objective functions.

The quality of a partition is evaluated using multiple architectural objectives, such as structural modularity, coupling, interface complexity, and balance in microservice size. These objectives are inherently conflicting, leading to a multi-objective optimization problem in which solutions approximate a Pareto front rather than a single optimum.

In addition to optimization objectives, the proposed formulation includes a database ownership constraint. Given a monolithic application of T database tables and a mapping $A : C \rightarrow 2^T$ indicating for each class the set of tables it accesses, for any table $t \in T$, all classes accessing t must be assigned to the same microservice.

4 Proposed Approach

4.1 Solution representation

The proposed representation models candidate solutions as partitions of the class set, where each individual corresponds to a decomposition obtained by grouping classes into k unlabeled microservices. According to the definitions presented earlier, given a set of classes $C = 0, 1, \dots, N - 1$, a solution is defined as a partition $P = M_1, M_2, \dots, M_k$ of C , in which each subset M_i represents a microservice. This representation ensures that each class is assigned to a single microservice.

Under this formulation, microservice labels and subset ordering are irrelevant given that two solutions are considered equivalent if they represent the same partition of classes. This canonical representation eliminates symmetric solutions and ensures a one-to-one correspondence between genotypes and distinct decompositions.

4.2 Objectives

The quality of a candidate decomposition is evaluated using the following evaluation metrics, classically employed in monolith decomposition techniques [13, 14].

Structural Modularity (SM). SM rewards high cohesion within microservices while penalizing coupling across microservices:

$$SM = \frac{1}{k} \sum_{i=1}^k \frac{|E(M_i, M_i)|}{|M_i|^2} - \frac{1}{\frac{k(k-1)}{2}} \sum_{i \neq j} \frac{|E(M_i, M_j)|}{2|M_i||M_j|} \quad (1)$$

Higher values of SM indicate decompositions with improved modular structure.

Interface Number (IN). IN measures the number of inter-microservice dependencies:

$$IN = \frac{1}{k} \sum_{i=1}^k \sum_{\substack{j=1 \\ j \neq i}}^k |E(M_i, M_j)| \quad (2)$$

Inter-Call Percentage (ICP). ICP quantifies the proportion of method calls crossing microservice boundaries:

$$ICP = \frac{\sum_{i=1}^k \sum_{\substack{j=1 \\ i \neq j}}^k icp(M_i, M_j)}{\sum_{i=1}^k \sum_{j=1}^k icp(M_i, M_j)} \quad (3)$$

where

$$icp(M_i, M_j) = \sum_{c_k \in M_i} \sum_{c_l \in M_j} (\log(calls(c_k, c_l)) + 1) \quad (4)$$

Non-Extreme Distribution (NED). NED promotes balanced microservice sizes by penalizing decompositions composed of very small or very large services. Given the total number of classes N , acceptable service sizes are defined as a fraction of N , via the predefined bounds l and u , resulting in the objective:

$$NED = 1 - \frac{|\{M_i \mid l < |M_i| < u\}|}{k} \quad (5)$$

Optimization goal. The optimization problem is defined as the maximization of SM and the minimization of IN, ICP, and NED. In addition, a hard constraint enforces database isolation, requiring that no two microservices access the same database tables. Solutions violating this constraint are considered infeasible and discarded.

4.3 Genetic operators

The following genetic operators are employed :

Mutation. Mutation relocates a single class from one subset (microservice) to another. A source subset containing more than one class is randomly selected, and one of its classes is relocated to a randomly selected destination subset.

Crossover. Crossover creates new decompositions by combining randomly selected subsets from two parent individuals, ensuring that each class is assigned to exactly one subset or microservice in each resulting decomposition.

Selection. Selection is performed using the NSGA-III mechanism, which combines non-dominated sorting with reference-point-based diversity preservation to maintain a well-distributed approximation of the Pareto front.

5 Experimental Setup

The proposed approach was evaluated on JPetStore [9] and AcmeAir [17], two Java-based monolithic applications commonly used in the literature on microservice decomposition. The analyzed JPetStore version comprises 24 classes and 13 database tables, while AcmeAir contains 31 classes and 4 database tables. For both systems, class-to-class and class-to-table dependencies were extracted through static analysis. The resulting dependency information was used to construct the graphs required for the decomposition process.

All experiments were conducted using the DEAP evolutionary computation framework [5]. The NSGA-III algorithm was employed with the partition-based representation and operators described in Section 4.3. The stopping criterion was a fixed number of generations. For both case studies, the hyperparameter values of this algorithm were selected by using a Sobol-based sensitivity analysis with performance measured by hypervolume (HV). Based on this analysis, the selected values for λ (population size), μ (offspring number), p_m (mutation probability), p_c (crossover probability), and number of generations were 300, 120, 0.75, 0.25 and 200, respectively.

For each case study, the algorithm was executed using 30 independent runs with fixed random seeds to account for the stochastic nature of evolutionary search. Results are summarized using the median and interquartile range (IQR) of the objective values across runs. Prior to aggregation, objective values were normalized to ensure comparability across metrics with different scales. The obtained Pareto front approximations are evaluated using the objective functions defined in Section 4.2.

The proposed approach was compared against several baseline methods reported in the literature, including Bunch [15], CoGCN [3], FoSCI [7], MEM [14], and Mono2Micro [8]. Baseline results were taken from the corresponding published studies and evaluated using the same metrics. All experiments were executed on a machine equipped with an Intel Core i7 processor with 8 cores and 16 GB of RAM.

6 Results

6.1 JPetStore

For JPetStore, the proposed approach consistently produces Pareto fronts that capture explicit trade-offs among the considered objectives. The median objective values of the Pareto fronts computed across runs of the proposed method are NED = 0.310, SM = 0.195, ICP = 0.308, and IN = 2.042.

Compared to baseline methods (Table 1), the proposed approach achieves substantially lower ICP and higher SM while maintaining a moderate number of IN and NED. While M2M achieve better values for individual objectives such as IN or NED, this comes at the cost of substantially higher ICP or lower SM.

The variability across runs of the proposed approach remains limited for most objectives, with IQRs of [0.19, 0.20] for SM and [0.29, 0.33] for ICP, indicating stable behavior in terms of service modularity and inter-service interaction. NED exhibits moderate variability, with an IQR of [0.26, 0.33], reflecting alternative decompositions that slightly favor either service size balance or interaction minimization. The number of service interfaces also remains stable, with an IQR of [2.0, 2.17].

Table 1: Comparison of median objective values on JPetStore.

Method	NED ↓	SM ↑	ICP ↓	IN ↓
M2M	0.257	0.054	0.333	1.857
FoSCI	0.516	0.044	0.478	3.750
CoGCN	0.392	0.091	0.582	2.533
Bunch	0.667	—	0.477	7.948
MEM	1.000	0.124	0.434	3.429
Our approach	0.310	0.195	0.308	2.042

Table 2: Comparison of median objective values on AcmeAir.

Method	NED ↓	SM ↑	ICP ↓	IN ↓
M2M	0.429	0.072	0.527	3.375
FoSCI	0.407	0.095	0.706	4.375
CoGCN	0.250	0.038	0.444	2.846
Bunch	0.692	0.177	0.550	3.875
MEM	0.464	0.097	0.589	4.333
Our approach	0.333	0.162	0.191	1.250

6.2 AcmeAir

On AcmeAir, which exhibits a different balance between application logic and persistence concerns, the proposed approach also identifies meaningful trade-off solutions. The median objective values obtained across runs are NED = 0.333, SM = 0.162, ICP = 0.191, and IN = 1.25.

As shown in Table 2, the proposed approach substantially outperforms all baseline methods in terms of ICP and IN. While some baselines achieve higher SM or lower NED, these improvements are associated with significantly higher communication overhead.

The variability across runs remains limited for most objectives. The IQR for SM is narrow, with values in the interval [0.16, 0.17], indicating stable structural modularity. ICP exhibits moderate variability, with an IQR of [0.17, 0.23], reflecting alternative decompositions that slightly trade communication efficiency for improvements in other quality attributes. NED shows limited dispersion, with an IQR of [0.25, 0.33], while the number of service interfaces remains consistently low, with an IQR of [1.14, 1.42].

Overall, the results show that the proposed approach consistently identifies decompositions that adhere to the one-database-per-microservice principle, achieving low inter-service communication and a reduced number of interfaces across different systems. While some baseline methods obtain better values for individual structural objectives, these improvements typically come at the cost of higher communication overhead or interface complexity. In contrast, the proposed Pareto-based formulation explicitly exposes these trade-offs, enabling decompositions that balance structural modularity with interaction-related quality attributes.

7 Conclusions and Future Work

This paper presents a multi-objective evolutionary approach to monolith decomposition that explicitly incorporates database dependencies as constraints. The problem is formulated as a constrained partitioning task solved using NSGA-III to explore trade-offs among structural and interaction-related quality attributes.

The experimental evaluation on two monolithic applications shows that the proposed formulation supports architectural decision-making by enabling the analysis of multiple decomposition alternatives under explicit trade-offs, rather than producing a single solution optimized for a limited set of criteria.

As future work, we plan on extending the problem formulation with additional architectural constraints and objectives, especially those related to maintainability and scalability. Moreover, we will carry out experiments on larger monolithic applications in order to evaluate the applicability of our approach to real-world monoliths.

References

- [1] ABGAZ, Y., MCCARREN, A., ELGER, P., SOLAN, D., LAPUZ, N., BIVOL, M., JACKSON, G., YILMAZ, M., BUCKLEY, J., AND CLARKE, P. Decomposition of monolith applications into microservices architectures: A systematic review. *IEEE Transactions on Software Engineering PP* (08 2023), 1–32.
- [2] ASSUNÇÃO, W. K. G., COLANZI, T. E., CARVALHO, L., PEREIRA, J. A., GARCIA, A., DE LIMA, M. J., AND LUCENA, C. A multi-criteria strategy for redesigning legacy features as microservices: An industrial case study. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* (2021), pp. 377–387.
- [3] DESAI, U., BANDYOPADHYAY, S., AND TAMILSELVAM, S. Graph neural network to dilute outliers for refactoring monolith application. *CoRR abs/2102.03827* (2021).
- [4] EDELKAMP, S., AND SCHRÖDL, S. Chapter 13 - constraint search. In *Heuristic Search*, S. Edelkamp and S. Schrödl, Eds. Morgan Kaufmann, San Francisco, 2012, pp. 571–631.
- [5] FORTIN, F.-A., DE RAINVILLE, F.-M., GARDNER, M.-A., PARIZEAU, M., AND GAGNÉ, C. DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research 13* (jul 2012), 2171–2175.
- [6] FOWLER, M., AND LEWIS, J. Microservices.
- [7] JIN, W., LIU, T., CAI, Y., KAZMAN, R., MO, R., AND ZHENG, Q. Service Candidate Identification from Monolithic Systems Based on Execution Traces. *IEEE Transactions on Software Engineering 47*, 5 (2021), 987–1007. Conference Name: IEEE Transactions on Software Engineering.
- [8] KALIA, A. K., XIAO, J., KRISHNA, R., SINHA, S., VUKOVIC, M., AND BANERJEE, D. Mono2micro: a practical and effective tool for decomposing monolithic java applications to microservices. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2021), ACM.
- [9] LANDIS, J., RENOVATE[BOT], SHIMIZU, K., DEPENDABOT[BOT], MACARRON, E., DEPENDABOT-PREVIEW[BOT], AVEI, I., LU, B., RAMAN, B., GOTIMER, G., BOUTEMY, H., TRIPODI, S., TIZIANO, KING, Z., AND DONGXU. mybatis/jpetstore-6. <https://github.com/mybatis/jpetstore-6>, jan 16 2026.
- [10] LI, S., ZHANG, H., JIA, Z., LI, Z. E., ZHANG, C., LI, J., AND GAO, Q. A Dataflow-Driven Approach to Identifying Microservices from Monolithic Applications. *Journal of Systems and Software 157* (2019).
- [11] LIU, B., XIONG, J., REN, Q., TYSZBEROWICZ, S., AND YANG, Z. Log2ms: a framework for automated refactoring monolith into microservices using execution logs. In *2022 IEEE International Conference on Web Services (ICWS)* (2022), pp. 391–396.
- [12] MARTÍNEZ SAUCEDO, A., RODRÍGUEZ, G., AND YANNIBELLI, V. A novel metaheuristic approach to monolith decomposition into microservices. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (New York, NY, USA, 2025), GECCO '25 Companion, Association for Computing Machinery, p. 103–104.
- [13] MARTÍNEZ SAUCEDO, A., RODRÍGUEZ, G., GOMES ROCHA, F., AND DOS SANTOS, R. P. Migration of monolithic systems to microservices: A systematic mapping study. *Information and Software Technology 177* (2025), 107590.
- [14] MAZLAMI, G., CITO, J., AND LEITNER, P. Extraction of microservices from monolithic software architectures. In *2017 IEEE International Conference on Web Services (ICWS)* (2017), pp. 524–531.
- [15] MITCHELL, B., AND MANCORIDIS, S. On the automatic modularization of software systems using the bunch tool. *IEEE Transactions on Software Engineering 32*, 3 (2006), 193–208.
- [16] SELLAMI, K., OUNI, A., SAIED, M. A., BOUKTIF, S., AND MKAOUER, M. W. Improving microservices extraction using evolutionary search. *Information and Software Technology 151* (2022), 106996.
- [17] TOLLEFSON, D., AND SPYKER, A. [acmeair/acmeair](https://github.com/acmeair/acmeair). <https://github.com/acmeair/acmeair>, mar 9 2024.